OpenMP Code Explanation

CODE_DOCUMENTATION

OpenMP Code Documentation

This document provides comprehensive explanations of the syntax and logic for all C programs in the HPC codebase, including both main OpenMP examples and debugging exercises.

Table of Contents

1. Main OpenMP Examples
2. Bug Examples
3. OpenMP Directives Quick Reference

Main OpenMP Examples

1. omp_hello.c

Purpose: Basic introduction to parallel programming with OpenMP

Key OpenMP Directives:

- `#pragma omp parallel` - Creates a team of threads

Syntax:

```
#pragma omp parallel private(nthreads, tid)
{
    // Parallel code block executed by all threads
}
```

Logic Flow:

1. Declares variables `nthreads` (number of threads) and `tid` (thread ID)
2. Creates parallel region where each thread gets its own copy of private variables
3. Each thread obtains its unique ID using `omp_get_thread_num()`
4. Only the master thread (tid == 0) queries and prints the total number of threads
5. All threads synchronize at the end of the parallel region and terminate

Key Concepts:

- **private(variables)**: Each thread gets its own copy of these variables
- **Master thread**: Thread 0, typically handles initialization/finalization
- **Implicit barrier**: Automatic synchronization at the end of parallel region

---

2. omp_get_env.c

Purpose: Query and display OpenMP environment information

Key OpenMP Functions:

- `omp_get_num_procs()` - Number of processors available
- `omp_get_num_threads()` - Current number of threads
- `omp_get_max_threads()` - Maximum threads that can be created
- `omp_in_parallel()` - Check if currently in parallel region
- `omp_get_dynamic()` - Check if dynamic thread adjustment is enabled
- `omp_get_nested()` - Check if nested parallelism is supported

Syntax:

```
#pragma omp parallel private(nthreads, tid)
{
    tid = omp_get_thread_num();
    if (tid == 0) {
        // Only master thread executes this
        procs = omp_get_num_procs();
```

```
        nthreads = omp_get_num_threads();
        // ... etc
    }
}
```

Logic Flow:

1. Creates a parallel region with private thread ID variable
2. Master thread retrieves various environment properties
3. Prints system configuration information about OpenMP setup
4. Useful for debugging and understanding system capabilities

Key Concepts:

- **Environment queries**: Functions to understand system capabilities
- **Master-only execution**: Using `if (tid == 0)` to execute code once
- **System information**: Number of processors, thread limits, features

---

3. omp_matrix_multi.c

Purpose: Perform matrix multiplication using OpenMP parallelization

Key OpenMP Directives:

- `#pragma omp parallel shared(...) private(...)`
- `#pragma omp for schedule(static, chunk)`

Syntax:

```
#pragma omp parallel shared(a, b, c, nthreads, chunk) private
(tid, i, j, k)
{
    #pragma omp for schedule(static, chunk)
    for (i = 0; i < NRA; i++) {
        // Parallel loop - iterations distributed among threa
ds
```

```
    }
}
```

Data Structures:

- Matrix A: 62 rows × 15 columns
- Matrix B: 15 rows × 7 columns
- Matrix C: 62 rows × 7 columns (result)

Logic Flow:

1. Initializes three matrices with predefined dimensions
2. Sets chunk size to 10 for work distribution
3. Creates parallel region with:
    - **shared**: Arrays and configuration visible to all threads
    - **private**: Each thread has its own copies of loop indices and thread ID
4. Master thread prints initialization message
5. Distributes three initialization loops across threads using static scheduling
6. Performs matrix multiplication:
    - Outer loop parallelized across threads
    - Each thread handles assigned rows
    - Print statements show which thread processes which row
7. Prints final result matrix

Key Concepts:

- **Matrix multiplication formula**: C[i][j] = Σ(A[i][k] × B[k][j])
- **Shared vs. Private**: Large data structures shared; loop variables private
- **Static scheduling with chunks**: Predetermined distribution of iterations (chunk size = 10)
- **Nested loops**: Only outer loop parallelized for efficiency

---

4. omp_workshare.c

Purpose: Demonstrate dynamic scheduling of loop work distribution

Key OpenMP Directives:

- `#pragma omp parallel shared(...) private(...)`
- `#pragma omp for schedule(dynamic, chunk)`

Syntax:

```
#pragma omp for schedule(dynamic, chunk)
for (i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
    printf("Thread%d: c[%d]=%f\n", tid, i, c[i]);
}
```

Data:

- Chunk size: 10
- Array size: 100 elements

Logic Flow:

1. Initializes three arrays (a, b, c) with 100 elements
2. Creates parallel region
3. Master thread prints number of threads
4. Each thread prints startup message
5. Distributes loop work **dynamically**:
    - Threads request work chunks of size 10 as they become available
    - Provides better load balancing than static scheduling
6. Each thread prints which elements it processes
7. Performs element-wise addition: c[i] = a[i] + b[i]

Key Concepts:

- **Dynamic scheduling**: Threads grab work at runtime when idle

- **Load balancing**: Better for irregular work distribution
- **Chunk size (10)**: Number of iterations per work request
- **Output ordering**: Varies between runs due to dynamic nature

---

5. omp_workshare2.c

Purpose: Demonstrate sections construct for distributing distinct work blocks

Key OpenMP Directives:

- `#pragma omp parallel shared(...) private(...)`
- `#pragma omp sections nowait`
- `#pragma omp section`

Syntax:

```
#pragma omp sections nowait
{
    #pragma omp section
    {
        // Section 1 - one or more threads work here
    }

    #pragma omp section
    {
        // Section 2 - one or more threads work here
    }
}
```

Logic Flow:

1. Initializes four arrays (a, b, c, d)
2. Creates parallel region
3. Distributes work into distinct sections (not iterations):
    - **Section 1**: Computes c[i] = a[i] + b[i] (addition)

- **Section 2**: Computes d[i] = a[i] × b[i] (multiplication)

4. **nowait** clause: Threads don't wait for all sections to complete
5. Different threads may execute different sections concurrently
6. Each section prints its results

Key Concepts:

- **Sections vs. For**: Sections divide distinct code blocks; For divides iterations
- **nowait**: Removes implicit barrier between sections
- **Data sharing**: Arrays shared; local variables private
- **Work partitioning**: Different computation types assigned to different threads

---

6. omp_reduction.c

Purpose: Demonstrate reduction operation for combining thread results

Key OpenMP Directives:

- `#pragma omp parallel for reduction(+:sum)`

Syntax:

```
#pragma omp parallel for reduction(+:sum)
for (i = 0; i < n; i++)
    sum = sum + (a[i] * b[i]);
```

Logic Flow:

1. Initializes two arrays (a, b) with 100 elements
2. Creates parallel for loop with reduction operation
3. **Reduction syntax**: `reduction(operator:variable)`
   - **Operator**: `+` for addition
   - **Variable**: `sum` is combined across threads
4. Each thread calculates partial dot product on its subset of data
5. Private copy of `sum` created for each thread

6. Final results from all threads automatically combined using `+` operator
7. Prints final sum value

Key Concepts:

- **Reduction**: Combines results from parallel threads back to single value
- **Private copies**: Each thread has its own `sum` variable
- **Automatic combination**: OpenMP handles combining results
- **Dot product**: Mathematical operation: Σ(a[i] × b[i])
- **Operators**: `+`, , , `&`, `|`, `^`, `&&`, `||`

---

7. omp_orphan.c

Purpose: Demonstrate orphaned directives and scoping within function calls

Key OpenMP Directives:

- `#pragma omp for reduction(+ : sum)` (orphaned directive)

Syntax:

```
float dotprod() {
    int i, tid;
    tid = omp_get_thread_num();
#pragma omp for reduction(+ : sum)// Orphaned - not directly
in parallel region
    for (i = 0; i < VECLEN; i++) {
        sum = sum + (a[i] * b[i]);
    }
}

int main() {
    ...
#pragma omp parallel
    dotprod();  // Function called from parallel region
}
```

Global Variables:

- `float a[VECLEN]` - First vector (global)
- `float b[VECLEN]` - Second vector (global)
- `float sum` - Accumulation variable (global)

Logic Flow:

1. Defines function `dotprod()` with an orphaned `for` directive
2. Orphaned directive: `#pragma omp for` appears inside function, not directly after `#pragma omp parallel`
3. Main function initializes global arrays with values 0.0 through 99.0
4. Creates parallel region and calls the function
5. The for directive works because function is called from within parallel region
6. Reduction operation combines partial sums from all threads
7. Prints final dot product result

Key Concepts:

- **Orphaned directives**: Directives inside function calls from parallel region
- **Global scope**: Variables a, b, sum are global
- **Reduction with globals**: Critical for correctness
- **Function composition**: Allows modular parallel code
- **Implicit scoping**: Reduction variable accessible across scope boundaries

---

8. serial_pi_calc.c

Purpose: Calculate π using Monte Carlo dartboard algorithm (serial version)

Key Concepts:

- Random number generation
- Geometric probability
- Iterative averaging

Syntax:

```
double dboard(int darts) {
    // Generate random points in [-1, 1] × [-1, 1] square
    // Count how many fall within unit circle
    // Estimate π = 4 × (points in circle) / (total points)
}
```

Algorithm Details:

The dartboard algorithm:

1. Generate random points (x, y) in square [-1, 1] × [-1, 1]
2. Check if point falls in unit circle: $x^2 + y^2 \leq 1$
3. Ratio of points in circle to total points approximates π/4
4. Therefore: π ≈ 4 × (score / darts)

Logic Flow:

1. Sets seeds for random number generation
2. Performs 100 rounds of calculation
3. Each round throws 10,000 darts
4. For each dart:
    - Generates random x and y coordinates in [-1, 1]
    - Tests if point is within circle radius
    - Increments score if inside circle
5. Calculates π approximation: π = 4 × score / darts
6. Computes running average across all rounds
7. Prints results after each round
8. Improves accuracy with more iterations

Key Concepts:

- **Monte Carlo method**: Statistical sampling for area estimation

- **Geometric approximation**: Circle area / square area = π/4
- **Iterative averaging**: Progressively refines result
- **Random sampling**: 10,000 darts per round
- **Output precision**: Shows π approximation converging to true value

---

Bug Examples

These files contain intentional bugs for learning and debugging practice. Each demonstrates a common OpenMP pitfall.

Bug 1: bug1.c

Issue: Missing `reduction` clause with shared variable

Code Pattern:

```
#pragma omp parallel for shared(a, b, c, chunk) private(i, ti
d)
{
    tid = omp_get_thread_num();
    for (i = 0; i < N; i++) {
        c[i] = a[i] + b[i];  // Simple element-wise operation
    }
}
```

Problem:

- Each thread accesses the same loop variable `i` as private
- Attempts to use `tid` inside loop but not properly scoped
- Main issue: **Schedule and chunking not specified** - implicit static scheduling

Logic:

1. Initializes three arrays
2. Attempts parallelized vector addition
3. **Bug**: Missing or incorrect schedule specification

Learning Point:

- Explicit schedule clauses prevent ambiguity
- Loop variable scope matters in OpenMP

Bug 2: bug2.c

Issue: Race condition with unprotected shared variable

Code Pattern:

```
#pragma omp parallel for shared(a, b, c, chunk) private(i, ti
d)\
    schedule(static, chunk)
{
    tid = omp_get_thread_num();
    for (i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
        printf("tid=%d i=%d c[i]=%f\n", tid, i, c[i]);
    }
}
```

Problem:

- Uses `#pragma omp parallel for` incorrectly by including a code block with braces
- Syntax error: parallel for should apply directly to for loop

Incorrect Syntax:

```
#pragma omp parallel for ... {
    // Should not have explicit braces here
}
```

Correct Syntax:

```
#pragma omp parallel for ...
    for (i = 0; i < N; i++) {
```

```
        // Loop body
    }
```

Learning Point:

- Proper OpenMP directive syntax is critical
- Braces placement affects interpretation

Bug 3: bug3.c

Issue: Incorrect variable initialization in sections with barriers

Code Pattern:

```
#pragma omp parallel private(c, i, tid, section)
{
    // ...
#pragma omp sections nowait
    {
#pragma omp section
        {
            section = 1;
            for (i = 0; i < N; i++)
                c[i] = a[i] * b[i];
        }
#pragma omp section
        {
            section = 2;
            for (i = 0; i < N; i++)
                c[i] = a[i] + b[i];
        }
    }
}
```

Problem:

- Variable `c` is private to each thread

- Different threads work on different copies of `c`
- Results from one section not visible to another
- `nowait` removes synchronization after sections

Logic Issues:

1. Array `c` declared private
2. Section 1: computes multiplication into private `c`
3. Section 2: computes addition into different private `c`
4. Each section operates on its own copy
5. Results depend on thread assignments, unpredictable

Learning Point:

- Variable scoping determines data visibility
- Private variables in parallel regions are thread-local
- Shared data needed if results must be combined

Bug 4: bug4.c

Issue: Private array too large for stack memory

Code Pattern:

```
#pragma omp parallel shared(nthreads) private(i, j, tid, a)
{
    // Each thread: double a[1048][1048] on stack
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            a[i][j] = tid + i + j;
}
```

Problem:

- Array `a` is 1048 × 1048 = ~8 million doubles
- Each thread gets **private copy** = ~64 MB per thread

- With 8 threads: ~512 MB on stack
- Stack overflow likely
- Very inefficient memory usage

Memory Analysis:

```
Array size: 1048 × 1048 doubles
Memory per thread: 1048² × 8 bytes ≈ 8.8 MB
Total with 8 threads: 8.8 × 8 ≈ 70 MB
Stack limit: typically 1-8 MB per thread
Result: Program crash
```

Logic:

1. Each thread initializes large private array
2. Fills array with values based on thread ID + position
3. Uses minimal data - initialization unnecessary
4. Despite logic being sound, memory model is broken

Learning Point:

- Private data allocated on stack per thread
- Large private arrays cause memory problems
- Use static allocation or make shared when possible

Bug 5: bug5.c

Issue: Deadlock with locks in nested sections

Code Pattern:

```
#pragma omp parallel shared(a, b, nthreads, locka, lockb) pri
vate(tid)
{
    #pragma omp sections nowait
    {
```

```
        #pragma omp section
        {
            omp_set_lock(&locka);
            // ... work with a ...
            omp_set_lock(&lockb);  // Wait for lockb
            // ... work with b ...
            omp_unset_lock(&lockb);
            omp_unset_lock(&locka);
        }
        #pragma omp section
        {
            omp_set_lock(&lockb);
            // ... work with b ...
            omp_set_lock(&locka);  // Wait for locka
            // ... work with a ...
            omp_unset_lock(&locka);
            omp_unset_lock(&lockb);
        }
    }
}
```

Problem - Classic Deadlock:

1. Section 1 acquires lock A, waits for lock B
2. Section 2 acquires lock B, waits for lock A
3. Both sections block forever - **deadlock**

Deadlock Condition:

```
Thread 1: Lock A → (waiting for B)
Thread 2: Lock B → (waiting for A)
Neither can proceed
```

Logic Sequence:

1. Two sections attempt concurrent execution

2. Section 1: locks a, then attempts to lock b
3. Section 2: locks b, then attempts to lock a
4. Circular wait: deadlock occurs
5. Program hangs indefinitely

Learning Point:

- Lock ordering prevents deadlocks
- Acquire locks in same order everywhere
- Avoid circular dependencies in locking

Bug 6: bug6.c

Issue: Uninitialized reduction variable in function scope

Code Pattern:

```
float dotprod() {
    int i, tid;
    float sum;  // LOCAL variable - NOT initialized

    tid = omp_get_thread_num();
#pragma omp for reduction(+ : sum)
    for (i = 0; i < VECLEN; i++) {
        sum = sum + (a[i] * b[i]);  // sum contains garbage
    }
}

int main() {
    float sum;  // DIFFERENT variable from function sum
    sum = 0.0;

#pragma omp parallel shared(sum)// Sharing main's sum
        dotprod();  // Uses local sum!
```

```
    printf("Sum =%f\n", sum);  // sum is uninitialized!
}
```

Problem:

1. Function `dotprod()` declares local `float sum`, uninitialized
2. Main function declares separate `float sum = 0.0`
3. Main shares its `sum`, but function uses local `sum`
4. Local `sum` contains garbage values
5. Reduction operation on garbage + garbage = garbage result
6. Main's `sum` never modified - remains uninitialized
7. Print shows garbage value

Variable Scope Issue:

- `main()`: `float sum` (one variable)
- `dotprod()`: `float sum` (different variable, shadows main's)
- Reduction applies to function's local uninitialized `sum`
- No communication between the two `sum` variables

Logic Failure:

1. Local sum initialized with undefined data
2. Reduction accumulates undefined + vector products
3. Result completely unreliable
4. Main's sum never used or updated
5. Output shows memory garbage

Learning Point:

- Variable scope shadowing causes errors
- Uninitialized variables in reduction are critical bugs
- Global scope needed for variables shared across functions

- Reduction clause applies to variable in current scope

OpenMP Directives Quick Reference

Parallel Region Directives

Directive	Purpose	Syntax
`parallel`	Fork multiple threads	`#pragma omp parallel`
`parallel for`	Parallelize loop iterations	`#pragma omp parallel for`
`parallel sections`	Parallelize distinct code blocks	`#pragma omp parallel sections`

Work-Sharing Directives

Directive	Purpose	Syntax
`for` / `do`	Distribute loop iterations	`#pragma omp for`
`sections`	Distribute distinct work blocks	`#pragma omp sections`
`section`	Define section within sections	`#pragma omp section`
`single`	Execute code by single thread	`#pragma omp single`

Variable Scope Clauses

Clause	Meaning	Example
`shared(var)`	Variable visible to all threads	`shared(a, b, total)`
`private(var)`	Each thread gets own copy	`private(i, j, temp)`
`firstprivate(var)`	Private, initialized from original	`firstprivate(count)`
`lastprivate(var)`	Private, final value copied out	`lastprivate(result)`
`reduction(op:var)`	Combine results across threads	`reduction(+:sum)`

Synchronization Directives

Directive	Purpose	Syntax
`barrier`	Wait for all threads	`#pragma omp barrier`
`critical`	Only one thread at a time	`#pragma omp critical`

Directive	Purpose	Syntax
`atomic`	Atomic operation	`#pragma omp atomic`
`master`	Only master thread	`#pragma omp master`

Schedule Clauses

Schedule Type	Behavior	Use Case
`schedule(static)`	Fixed chunks predetermined	Regular workload
`schedule(dynamic, chunk)`	Chunks requested at runtime	Irregular workload
`schedule(guided)`	Decreasing chunk sizes	Load balancing
`schedule(auto)`	Implementation decides	Compiler optimization

Reduction Operators

- Addition: `reduction(+:var)`
- Multiplication: `reduction(*:var)`
- Bitwise AND: `reduction(&:var)`
- Bitwise OR: `reduction(\|:var)`
- Bitwise XOR: `reduction(^:var)`
- Logical AND: `reduction(&&:var)`
- Logical OR: `reduction(\|\|:var)`

Important Runtime Functions

Function	Returns
`omp_get_thread_num()`	Current thread ID (0 to n-1)
`omp_get_num_threads()`	Total threads in team
`omp_get_max_threads()`	Maximum threads available
`omp_get_num_procs()`	Number of processors
`omp_in_parallel()`	1 if in parallel region, else 0
`omp_set_num_threads(n)`	Set thread count
`omp_get_dynamic()`	Dynamic thread adjustment enabled?

Function	Returns
`omp_get_nested()`	Nested parallelism supported?
`omp_init_lock(lock)`	Initialize lock
`omp_set_lock(lock)`	Acquire lock
`omp_unset_lock(lock)`	Release lock

Summary

Main Examples (omp_*.c)

These files demonstrate core OpenMP concepts:

1. **omp_hello.c** - Basic parallelization
2. **omp_get_env.c** - System queries
3. **omp_matrix_multi.c** - Data parallelism with loops
4. **omp_workshare.c** - Dynamic scheduling
5. **omp_workshare2.c** - Sections for distinct tasks
6. **omp_reduction.c** - Combining results across threads
7. **omp_orphan.c** - Directives in function calls
8. **serial_pi_calc.c** - Monte Carlo algorithm baseline

Bug Examples (bugs/bug*.c)

These demonstrate common pitfalls:

1. **bug1.c** - Missing schedule specification
2. **bug2.c** - Syntax errors with parallel for
3. **bug3.c** - Incorrect variable scoping
4. **bug4.c** - Memory overflow with private arrays
5. **bug5.c** - Deadlock with locks
6. **bug6.c** - Uninitialized reduction variables

Understanding these examples provides comprehensive knowledge of OpenMP parallel programming patterns and common mistakes to avoid.